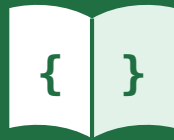

Modül 16 — Async, Zamanlama ve Spring Batch

Ders Notları

Spring Boot Edu



2026-09-24

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Üç Tür “Şimdi Değil”	2
2.2 Chunk Tabanlı İşleme	2
2.3 Job, Job Instance, Job Execution	3
3. Adım Adım Örnekler	3
3.1 @Async ve CompletableFuture	3
3.2 @Scheduled	5
3.3 Bir Spring Batch Job’ı: Reader, Processor, Writer	6
3.4 Step: Chunk’lar, Skip ve Retry	8
3.5 Job Parametreleri ve Yeniden Başlatma	9
3.6 Test	10
4. Sık Yapılan Hatalar ve En İyi Pratikler	10
5. Özet	11
6. İleri Okuma	11

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Yavaş çağrılar `@Async` ve `CompletableFuture` ile paralel çalıştırmak
- `@Async` ve `@Scheduled`'in virtual thread'lerde çalışmasını sağlamak
- İşleri `@Scheduled` (sabit gecikme ve cron) ile zamanlamak ve birden çok örnekte neyin değiştiğini bilmek
- Bir Spring Batch 6 job'ı kurmak: reader, processor, writer, chunk'lar
- Bir job'ı dayanıklı yapmak: bozuk veriyi atlamak, geçici hataları yeniden denemek, çöküşten sonra yeniden başlatmak
- Async kodu, zamanlamaları ve batch job'larını test etmek

Ön koşullar: Modül 06 (PostgreSQL, transaction'lar) · **Tahmini süre:** 5 saat · **Docker gerekir**

2. Kavramlar

2.1 Üç Tür “Şimdi Değil”

Araç	Çalışma zamanı	Tipik kullanım
<code>@Async</code>	şimdi, ama başka bir thread'de; çağırın beklemez	üç yavaş servisi aynı anda çağırmak
<code>@Scheduled</code>	daha sonra, sabit aralıklarla veya belirli saatlerde	30 saniyede bir temizlik, 02:00'de rapor
Spring Batch	büyük miktarda veri, chunk'lar hâlinde, yeniden başlatılabilir	her gece 2 milyon CSV satırını içe aktarmak

2.2 Chunk Tabanlı İşleme

Bir batch adımı (step) öğeleri **chunk**'lar hâlinde okur, işler ve yazar. Her chunk bir transaction'dır:

```
10x oku → 10x işle → 10'unu yaz → COMMIT   (chunk 1: öge 1–10)
10x oku → 10x işle → 10'unu yaz → COMMIT   (chunk 2: öge 11–20)
...
```

Her commit'ten sonra Spring Batch, adımın nereye kadar geldiğini **job repository**'sine (veritabanı tabloları) kaydeder. Yeniden başlatmayı mümkün kılan budur: Başarısız bir job, son commit edilen chunk'tan sonra devam eder.

2.3 Job, Job Instance, Job Execution

Terim	Anlamı	Örnek
Job	tanım	importBooksJob
Job instance	job + tanımlayıcı parametreler	input.file=books-2026-09.csv için importBooksJob
Job execution	bir instance'ı çalıştırma denemesi	pazartesi başarısız olan çalışma, salı günkü yeniden başlatma

Tamamlanmış bir job instance tekrar çalışamaz. Başarısız bir instance ise gerektiği kadar yeniden başlatılabilir.

3. Adım Adım Örnekler

Docker açıkken uygulamayı başlatın. PostgreSQL kök dizindeki `compose.yaml` dosyasından başlar:

```
./mvnw -pl modules/16-async-scheduling-batch/lesson spring-boot:run
```

Tur çıktısı:

```
== 3.1 @Async on virtual threads
5 price calls of 200 ms: total 424.90 in 202 ms
== 3.3 A Spring Batch job
status COMPLETED: read 7, written 5, skipped 2
== 3.5 The same parameters again
refused: A job instance already exists and is complete for identifying parameters=...
```

3.1 @Async ve CompletableFuture

```
@Service
public class PriceClient {

    private static final Map<String, BigDecimal> PRICES = Map.of(
        "9780134685991", new BigDecimal("89.90"), "9781617297571", new
        ↪ BigDecimal("95.00"),
        "9780321336781", new BigDecimal("55.00"), "9781449373320", new
        ↪ BigDecimal("110.00"),
        "9781492078005", new BigDecimal("75.00"));

    private final Set<String> threadsSeen = ConcurrentHashMap.newKeySet();

    @Async // runs on the task
    ↪ executor, not on the caller
    public CompletableFuture<BigDecimal> priceOf(String isbn) {
        Thread current = Thread.currentThread();
        threadsSeen.add((current.isVirtual() ? "virtual:" : "platform:") +
        ↪ current.getName());
        sleep(200); // the remote call
        BigDecimal price = PRICES.get(isbn);
        return price == null
            ? CompletableFuture.failedFuture(new NoSuchElementException("No price for
            ↪ " + isbn))
            : CompletableFuture.completedFuture(price);
    }
}
```

```
@Service
public class PriceAggregator {

    private final PriceClient prices;

    public PriceAggregator(PriceClient prices) {
        this.prices = prices; // another bean: calls
        ↪ go through the proxy
    }

    public BigDecimal totalFor(List<String> isbnns) {
        List<CompletableFuture<BigDecimal>> calls =
        ↪ isbnns.stream().map(prices::priceOf).toList(); // all started
        return calls.stream()
            .map(CompletableFuture::join) // now wait
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    }
}
```

- @EnableAsync özelliği açar. Bir @Async metot task executor'da çalışır ve çağırılan hemen bir CompletableFuture alır.

- PriceAggregator önce tüm çağrıları **başlatır**, sonra hepsini **bekler**: 200 ms'lik beş çağrı yaklaşık 200 ms sürer.
- `spring.threads.virtual.enabled=true` ile Boot'un executor'ı her görev için bir virtual thread başlatır. PriceAggregatorTest, çağrılarının virtual thread'lerde çalıştığını kontrol eder.

! Önemli

@Async, @Transactional gibi bir proxy üzerinden çalışır. **Aynı sınıftaki** bir metottan yapılan çağrı, çağırının thread'inde senkron çalışır. PriceAggregator ve PriceClient'in iki ayrı bean olmasının nedeni budur.

★ İpucu

Bir @Async metodun içindeki exception, future'ı hatayla tamamlar. Çağıranı onu `join()` veya `exceptionally(...)` ile görür. void döndüren @Async metotlarda ise bir AsyncUncaughtExceptionHandler dışında kimse onu görmez.

3.2 @Scheduled

Aralıklar yapılandırmadan gelir:

```
bookstore:
  snapshot:
    interval: 30s                                # @Scheduled(fixedDelayString = ...)
  report:
    cron: "0 0 2 * * *"                          # every night at 02:00 (second minute hour
↪ day month weekday)
```

```
@Component
public class StockSnapshotTask {

    private final AtomicInteger runs = new AtomicInteger();

    @Scheduled(fixedDelayString = "${bookstore.snapshot.interval}") // 30s, 100ms, ...
    ↪ from application.yaml
    void takeSnapshot() {
        runs.incrementAndGet(); // a real task
    ↪ would copy the stock levels
    }

    public int runs() {
        return runs.get();
    }
}
```

```

@Component
class NightlyReportTask {

    private static final Logger log = LoggerFactory.getLogger(NightlyReportTask.class);

    @Scheduled(cron = "${bookstore.report.cron}", zone = "Europe/Istanbul")
    void createReport() {
        log.info("nightly report created"); // with several
        ↪ instances: see ShedLock (3.2)
    }
}

```

Özellik	Anlamı
fixedDelay	önceki çalışma bittikten sonra bu kadar bekle
fixedRate	önceki çalışma hâlâ sürse bile her periyotta başla
cron	şu saatlerde: saniye dakika saat gün ay haftanın günü, ör. 0 0 2 * * * = her gün 02:00'de

- Cron job'larında her zaman zone belirtin: Sunucular ve container'lar çoğu zaman UTC'de çalışır.
- Bir cron ifadesi gece 2'yi bekleyerek değil, bir sonraki çalışmayı sorarak (`CronExpression.parse(...)` test edilir (`NightlyReportCronTest`)).

! Uyarı

`@Scheduled` uygulamanın **her** örneğinde çalışır. Üç örnekle gece raporu üç kez oluşturulur. Yaygın çözüm dağıtık bir kilittir: **ShedLock** veritabanında (veya Redis'te, ...) bir kilit satırı tutar ve görevi yalnızca kilidi alan örnek çalıştırır. Alternatif olarak bu tür işleri tek bir zamanlayıcıya taşıyın: bir Kubernetes CronJob'ı (modül 22) veya bir kez başlatılan bir Batch job'ı.

3.3 Bir Spring Batch Job'ı: Reader, Processor, Writer

Job repository PostgreSQL'de durur. Boot tablolarını oluşturur ve job'ları kendiliğinden başlatmaz (spring: altında):

```

batch:
  jdbc:

```

```

initialize-schema: always                # create BATCH_JOB_INSTANCE,
↪ BATCH_STEP_EXECUTION, ... in PostgreSQL
job:
  enabled: false                        # do not run jobs at startup; LessonTour
↪ starts them with parameters

```

Reader CSV satırlarını birbiri ardına okur. Step scope'ludur, bu yüzden dosyayı job parametrelerinden alır:

```

@Bean
@StepScope                                // a new reader per step run,
↪ with the job's parameters
FlatFileItemReader<BookLine> bookLineReader(@Value("#{jobParameters['input.file']}")
↪ Resource input) {
    return new FlatFileItemReaderBuilder<BookLine>()
        .name("bookLineReader")           // the key of its position in
        ↪ the ExecutionContext
        .resource(input)
        .linesToSkip(1)                   // the header line
        .delimited().names("isbn", "title", "price")
        .targetType(BookLine.class)      // records are supported
        .build();
}

```

Processor bir satırı doğrular ve dönüştürür. Sade Java'dır, bu yüzden BookLineProcessor-Test onu bir job olmadan test eder:

```

@Component
public class BookLineProcessor implements ItemProcessor<BookLine, ImportedBook> {

    private final ImportFaults faults;

    public BookLineProcessor(ImportFaults faults) {
        this.faults = faults;
    }

    @Override
    public ImportedBook process(BookLine line) {
        faults.check(line.isbn());
        if (!line.isbn().matches("\\d{13}")) {
            throw new InvalidBookLineException("invalid ISBN: " + line.isbn());
        }
        BigDecimal price = parsePrice(line.price());
        if (price.signum() < 0) {
            throw new InvalidBookLineException("negative price for " + line.isbn());
        }
    }
}

```

```

        return new ImportedBook(line.isbn(), line.title().trim(), price);
    }

```

Writer bütün bir chunk'ı tek bir JDBC batch'inde yazar. ON CONFLICT ... DO UPDATE onu idempotent yapar. Böylece aynı kitabı iki kez yazmak (ör. yeniden başlatmadan sonra) zarar vermez:

```

@Bean
JdbcBatchItemWriter<ImportedBook> importedBookWriter(dataSource) {
    return new JdbcBatchItemWriterBuilder<ImportedBook>()
        .dataSource(dataSource)
        .sql("""
            INSERT INTO imported_book (isbn, title, price) VALUES (:isbn, :title,
↪ :price)
            ON CONFLICT (isbn) DO UPDATE SET title = EXCLUDED.title, price =
↪ EXCLUDED.price,
                                                    imported_at = now()""") //
↪ idempotent: safe to rerun
        .itemSqlParameterSourceProvider(book -> new MapSqlParameterSource()
            .addValue("isbn", book.isbn()).addValue("title",
↪ book.title()).addValue("price", book.price()))
        .build();
}

```

3.4 Step: Chunk'lar, Skip ve Retry

```

@Bean
Step importStep(JobRepository jobRepository, PlatformTransactionManager
↪ transactionManager,
                FlatFileItemReader<BookLine> reader, BookLineProcessor processor,
                JdbcBatchItemWriter<ImportedBook> writer, ImportSkipLog skipLog) {
    return new StepBuilder("importStep", jobRepository)
        .<BookLine, ImportedBook>chunk(10) // 10 items per
        ↪ transaction
        .transactionManager(transactionManager)
        .reader(reader)
        .processor(processor)
        .writer(writer)
        .faultTolerant()
        .skip(InvalidBookLineException.class) // bad data: skip
        ↪ the item ...
        .skip(FlatFileParseException.class)
        .skipLimit(10) // ... but not more
        ↪ than 10 times
    }
}

```

```

        .retry(TransientDataAccessException.class) // a short
        ↪ database hiccup: try again
        .retryLimit(3)
        .skipListener(skipLog)
        .build();
    }

@Bean
Job importBooksJob(JobRepository jobRepository, Step importStep) {
    return new JobBuilder("importBooksJob", jobRepository)
        .start(importStep)
        .build();
}

```

- `chunk(10)`: transaction başına 10 öge. Büyük chunk'lar daha hızlıdır, küçük chunk'lar bir hatada daha az iş kaybettirir.
- **Skip**, asla düzelmeyecek bozuk veri içindir: Örnek dosyadaki bozuk ISBN ve negatif fiyat atlanır ve loglanır (`ImportSkipLog`). 10'dan fazla atlama job'ı başarısız yapar: Çok fazla bozuk veri genellikle yanlış bir dosya demektir.
- **Retry**, kısa süre erişilemeyen bir veritabanı gibi geçici hatalar içindir. Aynı öge 3 kereye kadar yeniden denenir.
- Spring Batch 6'da `chunk(int)` yeni `ChunkOrientedStepBuilder`'ı döndürür. Retry desteği Spring Framework 7'nin `retry` API'sini kullanır.

Bir çalışmadan sonra job repository'ye bakın:

```

docker compose exec postgres psql -U bookstore -c \
    "SELECT step_name, read_count, write_count, process_skip_count, commit_count, status
    ↪ FROM batching.batch_step_execution"

```

step_name	read_count	write_count	process_skip_count	commit_count	status
importStep	7	5	2	1	COMPLETED

3.5 Job Parametreleri ve Yeniden Başlatma

Job parametrelerle başlatılır. `input.file` job instance'ını tanımlar:

```

var parameters = new JobParametersBuilder()
    .addString("input.file", "classpath:import/books-2026-09.csv")
    .toJobParameters();
JobExecution execution = jobOperator.start(importBooksJob, parameters);

```

`ImportBooksJobTest`, job repository'nin neyi mümkün kıldığını gösterir:

1. **Tamamlanmış instance'lar iki kez çalışmaz.** Aynı parametreleri yeniden başlatmak `JobInstanceAlreadyCompleteException` fırlatır. Bu yüzden gecelik bir import, tarihi parametrelerine ekler.
2. **Başarısız instance'lar kaldıkları yerden devam eder.** Test 30 satır içe aktarır ve 25. satırda bir çöküş simüle eder. Chunk 1 ve 2 commit edilmiştir (20 satır) ve job FAILED olur. Problem giderildikten sonra `jobOperator.restart(execution)` yalnızca 21-30. satırları okur ve job 30 satırın tamamıyla COMPLETED biter.

3.6 Test

- `@Async`: Paralel çağrıların yaklaşık tek bir çağrı kadar sürdüğünü ölçün ve thread'i kontrol edin (`Thread.isVirtual()`).
- `@Scheduled`: Aralığı bir property ile kısaltın (`bookstore.snapshot.interval=100ms`) ve görev çalışana kadar `Awaitility` ile bekleyin.
- Spring Batch: `@SpringBatchTest`, `JobOperatorTestUtils`'i (job'ları ve tek tek step'leri başlatır) ve `JobRepositoryTestUtils`'i (testler bağımsız olsun diye eski çalışmaları siler) sağlar.

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! Dikkat

Bir batch writer'ı idempotent olmalıdır. Bir çöküşten sonra son chunk yeniden işlenir. Düz bir INSERT ile yeniden başlatma tekrarlanan anahtarlarda başarısız olur veya kopyalar oluşturur.

- **Yapın:** İlk sonucu beklemeden önce tüm `@Async` çağrıları başlatın. `priceOf(a).join(); priceOf(b).join();` yine sıralıdır.
- **Yapmayın:** Bir `@Async` metodu aynı sınıftan çağırmayın. Çağrı proxy'yi atlar ve senkron çalışır.
- **Yapın:** Cron zamanlamalarına açık bir saat dilimi verin ve onları birden çok örneğe karşı koruyun (`ShedLock` veya tek bir zamanlayıcı).
- **Yapmayın:** "Sonra tekrar dene" anlamına gelen exception'ları atlamayın, "asla" anlamına gelenleri yeniden denemeyin. Bozuk veriyi atlayın, geçici hataları yeniden deneyin.
- **Yapın:** Bir çalışmayı tanımlayan her şeyi (dosya, tarih) job parametrelerine koyun. Böylece tekrarlar ve yeniden başlatmalar öngörülebilir davranır.
- **Yapmayın:** Bütün bir dosyayı belleğe yüklemeyin. Reader'lar veriyi chunk chunk akıtır.

5. Özet

- `@Async` hemen bir `CompletableFuture` döndürür. Tüm çağrılar başlatın, sonra bekleyin: Beklemeler üst üste biner. Virtual thread'lerle görevdeki bloklama ucuzdur.
- `@Scheduled` sabit bir gecikme, sabit bir hız veya bir cron ifadesiyle çalışır. Birden çok örnekte `ShedLock` gibi bir kilit gerekir.
- Bir Spring Batch step'i chunk'lar hâlinde okur, işler ve yazar. Her chunk bir transaction'dır.
- Skip bozuk veri, retry geçici hatalar içindir. Job repository ilerlemeyi kaydeder.
- Job parametreleri bir job instance'ını tanımlar. Tamamlanmış instance'lar tekrar çalışmaz, başarısız olanlar son commit edilen chunk'tan sonra yeniden başlar.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Framework — Task Execution and Scheduling](#)
- [Spring Boot — Task Execution and Scheduling](#)
- [Spring Batch Reference · Chunk-oriented Processing](#)
- [Spring Boot — Batch Applications](#)
- [ShedLock](#)